ARTIFICIAL INTELLIGENCE LABORATORY RECORD

PROLOG PROGRAMS 17–33

Name: Mohammad ALEYAS

Register Number: 192521220

Course: AI

Course Code: CSA1702 – AI LAB EXPERIMENTS

Experiment 17: Sum the Integers from 1 to N

AIM

To write a Prolog program to calculate the sum of integers from 1 to N.

ALGORITHM / PROCEDURE

The base case states that the sum up to 0 is 0. The recursive rule adds N to the sum up to N-1.

PROGRAM —

program17.pl | SWI-Prolog source

```
sum_n(0, 0).
sum_n(N, Sum) :-
   N > 0,
   N1 is N - 1,
   sum_n(N1, S1),
   Sum is S1 + N.
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- sum_n(5, S).
S = 15.
```

SWI-Prolog | Terminal

```
?- ?- sum_n(10, S).
S = 55.
```

Experiment 18: Database with Name and Date of Birth

AIM

To create a simple Prolog database containing names and dates of birth and retrieve records using queries.

ALGORITHM / PROCEDURE

Facts form the database. A query can retrieve all records or the date of birth for one person.

PROGRAM —

program18.pl | SWI-Prolog source

```
% Facts: person(Name, DateOfBirth).
person(rahul, date(12, 5, 2003)).
person(priya, date(8, 11, 2002)).
person(aman, date(21, 1, 2004)).
```

```
person(sara, date(3, 7, 2003)).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- person(Name, DOB).
Name = rahul, DOB = date(12, 5, 2003) ;
Name = priya, DOB = date(8, 11, 2002) ;
...
```

SWI-Prolog | Terminal

```
?- ?- person(priya, DOB).
DOB = date(8, 11, 2002).
```

Experiment 19: Student–Teacher–Subject Code Database

AIM

To represent student, teacher and subject-code information in Prolog and query the relationships.

ALGORITHM / PROCEDURE

The predicates store students, teachers and subjects. The rule teaches/2 finds the teacher associated with a student's subject code.

PROGRAM —

program19.pl | SWI-Prolog source

```
% student(StudentName, SubjectCode).
student(arun, it101).
student(meena, it102).
student(rahul, it101).

% teacher(TeacherName, SubjectCode).
teacher(mr_kumar, it101).
teacher(ms_latha, it102).

% subject(SubjectCode, SubjectName).
subject(it101, programming).
subject(it102, database).

teaches(Teacher, Student) :-
    teacher(Teacher, Code),
    student(Student, Code).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- student(arun, Code).
Code = it101.
```

SWI-Prolog | Terminal

```
?- ?- teaches(Teacher, meena).
Teacher = ms_latha.
```

SWI-Prolog | Terminal (editable)

```
?- ?- subject(it101, Name).
Name = programming.
```

Experiment 20: Planets Database

AIM

To create a database of planets and retrieve their order and properties.

ALGORITHM / PROCEDURE

Each planet is stored as a fact with its position and broad type. inner_planet/1 is a rule that identifies the four inner planets.

PROGRAM —

program20.pl | SWI-Prolog source

```
% planet(Name, OrderFromSun, Type).
planet(mercury, 1, terrestrial).
planet(venus, 2, terrestrial).
planet(earth, 3, terrestrial).
planet(mars, 4, terrestrial).
planet(jupiter, 5, gas_giant).
planet(saturn, 6, gas_giant).
planet(uranus, 7, ice_giant).
planet(neptune, 8, ice_giant).

inner_planet(Name) :-
    planet(Name, Position, terrestrial),
    Position =< 4.
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- planet(Name, 3, Type).
Name = earth, Type = terrestrial.
```

SWI-Prolog | Terminal

```
?- ?- inner_planet(P).
P = mercury ; P = venus ; P = earth ; P = mars.
```

Experiment 21: Towers of Hanoi

AIM

To implement the Towers of Hanoi problem using recursion in Prolog.

ALGORITHM / PROCEDURE

For N disks, move N-1 disks to the auxiliary peg, move the largest disk to the destination, then move N-1 disks onto it.

PROGRAM —

program21.pl | SWI-Prolog source

```
hanoi(1, Source, Destination, _) :-
   write('Move disk from '), write(Source),
   write(' to '), write(Destination), nl.

hanoi(N, Source, Destination, Auxiliary) :-
   N > 1,
   N1 is N - 1,
   hanoi(N1, Source, Auxiliary, Destination),
   hanoi(1, Source, Destination, Auxiliary),
   hanoi(N1, Auxiliary, Destination, Source).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- hanoi(3, left, right, middle).
Displays the seven moves needed to transfer 3 disks from left to right.
```

Note: For 3 disks, the minimum number of moves is 2^3 − 1 = 7.

Experiment 22: Bird Can Fly or Cannot Fly

AIM

To determine whether a particular bird can fly using facts and rules.

ALGORITHM / PROCEDURE

The database lists birds and exceptions. Negation-as-failure (\+/1) makes can_fly/1 succeed only when the bird is not known to be flightless.

PROGRAM —

program22.pl | SWI-Prolog source

```
bird(sparrow).
bird(parrot).
bird(eagle).
bird(penguin).
bird(ostrich).

cannot_fly(penguin).
cannot_fly(ostrich).

can_fly(Bird) :-
   bird(Bird),
   \+ cannot_fly(Bird).
```

```
flight_status(Bird, 'can fly') :-
    can_fly(Bird).
flight_status(Bird, 'cannot fly') :-
    cannot_fly(Bird).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- flight_status(sparrow, Status).
Status = 'can fly'.
```

SWI-Prolog | Terminal

```
?- ?- flight_status(penguin, Status).
Status = 'cannot fly'.
```

SWI-Prolog | Terminal

```
?- ?- can_fly(X).
X = sparrow ; X = parrot ; X = eagle.
```

Experiment 23: Family Tree

AIM

To represent family relationships and derive parent, grandparent, sibling and ancestor relationships.

ALGORITHM / PROCEDURE

Basic family facts are used to derive relationships. The recursive ancestor rule follows parent links through any number of generations.

PROGRAM —

program23.pl | SWI-Prolog source

```
% parent(Parent, Child).
parent(john, peter).
parent(mary, peter).
parent(john, lisa).
parent(mary, lisa).
parent(peter, sam).
parent(anna, sam).

male(john).
male(peter).
male(sam).
female(mary).
female(lisa).
female(anna).

father(F, C) :- parent(F, C), male(F).
mother(M, C) :- parent(M, C), female(M).

grandparent(G, C) :-
    parent(G, P),
    parent(P, C).

sibling(X, Y) :-
    parent(P, X),
```

```
    parent(P, Y),
    X \= Y.

ancestor(A, D) :- parent(A, D).
ancestor(A, D) :-
    parent(A, X),
    ancestor(X, D).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- father(john, peter).
true.
```

SWI-Prolog | Terminal

```
?- ?- grandparent(john, sam).
true.
```

SWI-Prolog | Terminal

```
?- ?- sibling(peter, lisa).
true.
```

SWI-Prolog | Terminal

```
?- ?- ancestor(john, sam).
true.
```

Experiment 24: Diet Suggestion System Based on Disease

AIM

To demonstrate a simple rule-based diet suggestion system. This is an educational example, not medical advice.

ALGORITHM / PROCEDURE

The diet/2 facts map a condition to general food guidance. The query retrieves the matching suggestion. A real diet must be individualized with a qualified clinician.

PROGRAM —

program24.pl | SWI-Prolog source

```
diet(diabetes, 'Choose high-fibre foods; limit sugary drinks and refined sweets').
diet(hypertension, 'Prefer lower-sodium foods; limit heavily salted and processed foods').
diet(anemia, 'Include iron-rich foods such as beans and leafy greens; pair plant iron with vitamin C').
diet(obesity, 'Prefer balanced portions, vegetables, whole grains and regular meals').
diet(acidity, 'Try smaller meals and identify personal trigger foods').

suggest_diet(Disease, Advice) :-
    diet(Disease, Advice).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- suggest_diet(diabetes, Advice).
Advice = 'Choose high-fibre foods; limit sugary drinks and refined sweets'.
```

SWI-Prolog | Terminal

```
?- ?- suggest_diet(anemia, Advice).
Returns the iron-rich food suggestion.
```

Experiment 25: Monkey and Banana Problem

AIM

To model a simple Monkey-Banana problem in which the monkey moves, pushes a box, climbs it and gets the banana.

ALGORITHM / PROCEDURE

A state records the monkey position, box position, whether the monkey is on the floor or box, and whether it has the banana. The move/3 rules describe legal actions. The solve/2 predicate searches for a plan.

PROGRAM —

program25.pl | SWI-Prolog source

```
% State: state(MonkeyPosition, BoxPosition, MonkeyStatus, HasBanana).
% MonkeyStatus is on_floor or on_box.

move(state(middle, middle, on_box, no),
    grab,
    state(middle, middle, on_box, yes)).

move(state(P, P, on_floor, no),
    climb,
    state(P, P, on_box, no)).

move(state(P, B, on_floor, no),
    push(P, Q),
    state(Q, Q, on_floor, no)) :-
   adjacent(P, Q).

move(state(P, B, on_floor, no),
    walk(P, Q),
    state(Q, B, on_floor, no)) :-
   adjacent(P, Q).

adjacent(door, middle).
adjacent(middle, door).
adjacent(middle, window).
adjacent(window, middle).

solve(State, []) :-
   State = state(_, _, _, yes).
solve(State, [Action | Rest]) :-
   move(State, Action, NextState),
   solve(NextState, Rest).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- solve(state(door, window, on_floor, no), Plan).
Returns a sequence of actions that reaches a state with HasBanana = yes, if reachable under the listed moves.
```

Experiment 26: Fruit and Its Colour Using Backtracking

AIM

To display fruit-colour pairs and demonstrate Prolog backtracking.

ALGORITHM / PROCEDURE

Prolog returns one matching fact at a time. Pressing the semicolon asks Prolog to backtrack and find the next solution.

PROGRAM —

program26.pl | SWI-Prolog source

```
fruit_colour(apple, red).
fruit_colour(banana, yellow).
fruit_colour(grapes, green).
fruit_colour(orange, orange).
fruit_colour(blueberry, blue).

% Query fruit_colour(Fruit, Colour) and press ; for next answer.
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- fruit_colour(Fruit, Colour).
Fruit = apple, Colour = red ;
Fruit = banana, Colour = yellow ;
Fruit = grapes, Colour = green ; ...
```

SWI-Prolog | Terminal

```
?- ?- fruit_colour(banana, C).
C = yellow.
```

Experiment 27: Best First Search

AIM

To implement Best First Search using a priority list ordered by heuristic value (smaller H is preferred).

ALGORITHM / PROCEDURE

The frontier contains candidate paths. The path with the smallest heuristic score is expanded first. This is a teaching example; the graph facts provide heuristic values for destination nodes.

PROGRAM —

program27.pl | SWI-Prolog source

```
% graph(Node, Neighbour, HeuristicOfNeighbour).
graph(a, b, 4).
graph(a, c, 2).
graph(b, d, 5).
graph(c, d, 1).
graph(c, e, 6).
graph(d, goal, 0).
graph(e, goal, 0).
```

```
best_first(Start, Goal, Path) :-
   search([[Start]], Goal, RevPath),
   reverse(RevPath, Path).

search([[Goal | Rest] | _], Goal, [Goal | Rest]).
search([ [Node | Path] | OtherPaths], Goal, Result) :-
   findall([Next, Node | Path],
        (graph(Node, Next, _), \+ member(Next, [Node | Path])),
        NewPaths),
   append(OtherPaths, NewPaths, Combined),
   sort_paths(Combined, Sorted),
   search(Sorted, Goal, Result).

sort_paths(Paths, Sorted) :-
   map_list_to_pairs(path_score, Paths, Pairs),
   keysort(Pairs, SortedPairs),
   pairs_values(SortedPairs, Sorted).

path_score([Node | _], H) :-
   (graph(_, Node, H) -> true ; H = 0).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- best_first(a, goal, Path).
Returns a path from a to goal, choosing the frontier path with the lowest heuristic score available in this example.
```

Note: Best First Search is not guaranteed to find an optimal path in general. The heuristic and graph determine the search order.

Experiment 28: Medical Diagnosis

AIM

To demonstrate a basic rule-based diagnosis from symptoms. It is for learning only and must not be used for real diagnosis.

ALGORITHM / PROCEDURE

The diagnosis rules match symptom combinations. Symptoms can overlap, and this simplified knowledge base cannot establish a real medical diagnosis.

PROGRAM —

program28.pl | SWI-Prolog source

```
symptom(john, fever).
symptom(john, cough).
symptom(john, body_pain).

symptom(mary, sneezing).
symptom(mary, runny_nose).

symptom(ali, fever).
symptom(ali, rash).

diagnosis(Person, flu_like_illness) :-
   symptom(Person, fever),
   symptom(Person, cough),
   symptom(Person, body_pain).

diagnosis(Person, common_cold_like_illness) :-
```

```
    symptom(Person, sneezing),
    symptom(Person, runny_nose).

diagnosis(Person, needs_clinical_assessment) :-
    symptom(Person, fever),
    symptom(Person, rash).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- diagnosis(john, D).
D = flu_like_illness.
```

SWI-Prolog | Terminal

```
?- ?- diagnosis(mary, D).
D = common_cold_like_illness.
```

SWI-Prolog | Terminal

```
?- ?- diagnosis(ali, D).
D = needs_clinical_assessment.
```

Experiment 29: Forward Chaining

AIM

To demonstrate forward chaining: derive new facts from known facts until no more rules can be applied.

ALGORITHM / PROCEDURE

Forward chaining starts with known facts and applies rules to infer conclusions. This compact example derives a grandparent fact from two parent facts; it demonstrates one inference step rather than a full rule engine.

PROGRAM —

program29.pl | SWI-Prolog source

```
% Initial facts.
fact(parent(alex, bob)).
fact(parent(bob, chris)).

% Rule: if X is parent of Y and Y is parent of Z,
% then X is grandparent of Z.
derive(grandparent(X, Z)) :-
    fact(parent(X, Y)),
    fact(parent(Y, Z)).

% Query the rule-derived conclusion.
forward_chain(Conclusion) :-
    derive(Conclusion).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- forward_chain(grandparent(alex, chris)).
```

```
true.
```

SWI-Prolog | Terminal

```
?- ?- derive(X).
X = grandparent(alex, chris).
```

Experiment 30: Backward Chaining

AIM

To demonstrate backward chaining: prove a goal by finding rules whose conclusions match it, then prove their conditions.

ALGORITHM / PROCEDURE

When a query is asked, Prolog works backward from the goal. It searches for a matching rule and recursively tries to prove each condition in that rule's body.

PROGRAM —

program30.pl | SWI-Prolog source

```
parent(alex, bob).
parent(bob, chris).
parent(chris, dana).

grandparent(X, Z) :-
   parent(X, Y),
   parent(Y, Z).

ancestor(X, Y) :-
   parent(X, Y).
ancestor(X, Y) :-
   parent(X, Z),
   ancestor(Z, Y).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- grandparent(alex, chris).
true.
```

SWI-Prolog | Terminal

```
?- ?- ancestor(alex, dana).
true.
```

SWI-Prolog | Terminal

```
?- ?- ancestor(alex, Who).
Who = bob ; Who = chris ; Who = dana.
```

Experiment 31: Create a Web Blog Using WordPress

AIM

To create a simple WordPress blog and demonstrate the title tag, anchor tag and basic HTML elements.

ALGORITHM / PROCEDURE

The anchor tag <a> creates a hyperlink. Its href attribute specifies the destination and its title attribute provides advisory text. The h1 and h2 tags define headings. WordPress menus may differ slightly by version.

PROGRAM —

program31.pl | SWI-Prolog source

```
HTML to add in a WordPress Custom HTML block:

<h1>My AI Lab Blog</h1>
<h2>Introduction to Artificial Intelligence</h2>
<p>This blog contains notes and practical work on AI.</p>

<a href="https://www.swi-prolog.org/"
  title="Visit the SWI-Prolog website">
  Learn SWI-Prolog
</a>

<p><strong>Bold text</strong> and <em>italic text</em>.</p>

<!-- The title attribute appears as a tooltip in many browsers. -->
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- Procedure
1. Sign in to WordPress and create a new site or post.
2. Add a title such as My AI Lab Blog.
3. Insert a Custom HTML block.
4. Paste the HTML shown above.
5. Preview and publish the post.
6. Click the link to verify the anchor tag; hover over it to view its title tooltip.
```

Note: Paste the HTML into a WordPress Custom HTML block, then preview the page.

Experiment 32: Pattern Matching

AIM

To implement simple pattern matching in Prolog by matching a list pattern against a list.

ALGORITHM / PROCEDURE

append/3 splits the input list into a prefix, the matching pattern and a suffix. Unification checks whether the pattern appears consecutively.

PROGRAM —

program32.pl | SWI-Prolog source

```
% A pattern matches when its elements occur
% consecutively in the input list.
contains_pattern(Pattern, List) :-
    append(_, Tail, List),
    append(Pattern, _, Tail).

% Example patterns and data.
sample_list([a, b, c, d, e]).
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- contains_pattern([b, c], [a, b, c, d]).
true.
```

SWI-Prolog | Terminal

```
?- ?- contains_pattern([c, e], [a, b, c, d]).
false.
```

SWI-Prolog | Terminal

```
?- ?- sample_list(L), contains_pattern([c, d], L).
L = [a, b, c, d, e].
```

Experiment 33: Count the Number of Vowels

AIM

To write a Prolog program to count the vowels in a list of characters.

ALGORITHM / PROCEDURE

The program checks each character. If it is a vowel, it adds 1; otherwise it keeps the count unchanged. This version expects lowercase character atoms.

PROGRAM —

program33.pl | SWI-Prolog source

```
vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

count_vowels([], 0).

count_vowels([H | T], Count) :-
   vowel(H),
   count_vowels(T, Rest),
   Count is Rest + 1.

count_vowels([H | T], Count) :-
   \+ vowel(H),
   count_vowels(T, Count).

% Input is a list of lowercase characters.
```

REQUIRED QUERIES AND SAMPLE OUTPUT — SWI-PROLOG TERMINAL

SWI-Prolog | Terminal

```
?- ?- count_vowels([h, e, l, l, o], N).
N = 2.
```

SWI-Prolog | Terminal

```
?- ?- count_vowels([a, e, i, o, u], N).
N = 5.
```

SWI-Prolog | Terminal

```
?- ?- count_vowels([p, r, o, l, o, g], N).
N = 2.
```